

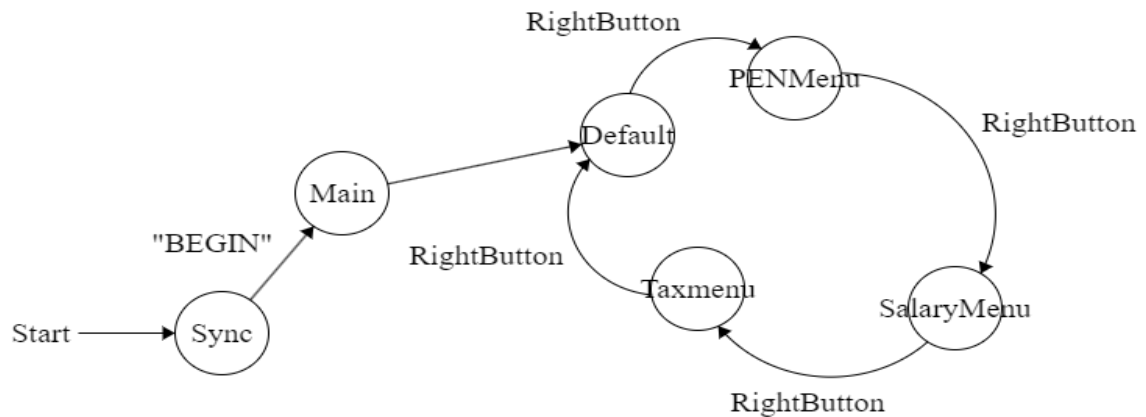
24COA202 Coursework

F421005

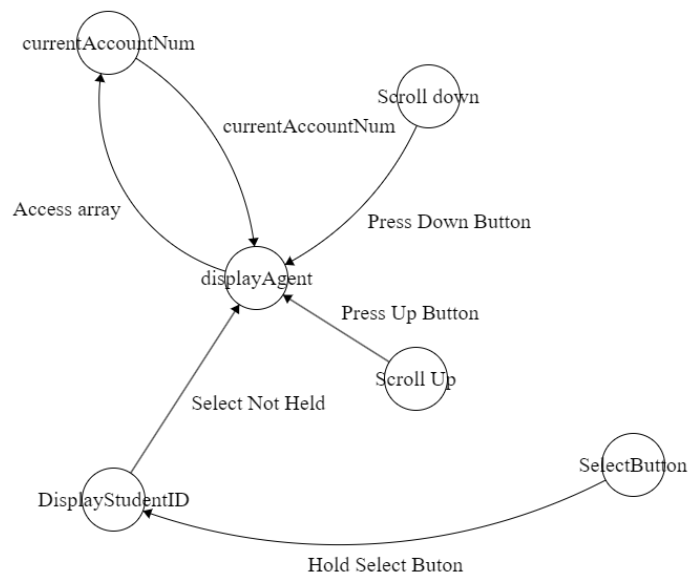
Semester 1

1 FSMs

Below shows a Simple Diagram of my FSM.



The FSM at the Centre of my system is the main and sync phases. The sync phase is responsible for the synchronization of the system to ensure it works and transitions to main via detection of the BEGIN string. The main loop handles the reading of inputs and the different protocols, such as the ADD where id and grade are turned from substrings to long and integer values, and the title is saved as a string. Aswell as all the other functions which require the id to be entered so the correct information can be changed, for example in the CJT protocol determining the next states for the sub-FSM. Where input via right button presses changes the menu state.



Above is a diagram on the Button presses for main state. The button functions in the code handle navigation and menu transitions within the system. Up button moves to the previous account stopping at the first account. And when in the pension menu will skip NPEN accounts. Similarly the down button moves to latter accounts stopping at the last. And when in PENMenu it will skip accounts with NPEN status. The select button when held will display student id and as well as the free memory.

2 Data structures

The 1 type of data structure I used was the structure for storing the information of the accounts, i chose to store salary as a double as this will ensure that large values are accepted and that when printed it automatically has it at 2 dp.

I also have many functions to update global variables such as my `taxCalculation()` function, which is a cluster of calculations to work out the tax that an account was due.

Another function was my `Pension()` function which simply gave a boolean value for whether an account was on pension or not. I would use this boolean value to code my pension menu to skip the accounts that were not on pension.

Another function I had was my `scrollingTitle()` function which simply scrolls the title on the LCD if its length exceeds 7 characters.

`displayAgent()` was another larger function in which I would display all the information on the LCD screen. I used this function with all my other functions.

Finally, I have an unfinished function the `displayMenu()` function which includes the enum for the menus in which the display would change depending on what menu was entered.

3 Debugging

```
. else if (currentMenu == PENMenu) {  
    lcd.clear();  
    //Serial.print("DEBUG: ENTER SAL MENU");
```

Lots of debugging time was spent during my attempts at the HCI extension.

```
if (inpStr.startsWith("GRD-")) {  
    //Serial.print("DEBUGG:      ");
```

Aswell as during the main phases when implementing the different protocols

one major problem I had was during the formatting for the titles, where it was only able to be alpha numeric with “_” as the only accepted special character. As initially anything was allowed. Then when I first started using the `isAlphaNumeric()` syntax I would always receive an error. Until checked every character using a for loop which worked after clearing the lcd

```
for (int i = 0; i < title.length(); i++) {  
    char specialCharacter = title[i];  
    if (!isalnum(specialCharacter) && specialCharacter != '_') {  
        validTitle = false;  
        Serial.print("ERROR: "); //error for invalid title  
        break;  
    }  
}
```

4 Reflection

I believe I have done well on this project, considering my experience. However, I do believe my codes could be greatly improved, as many repetitive lines of code were used during the start (when coding the different protocols) due to my lack of knowledge of the utility of defining functions. Also, I believe my code is rather messy due to the late addition of error messages. For example, during my implementation of the protocols, I skipped the error messages for if the wrong format was used or if the title was not alphanumeric with the only special character allowed being the underscore. As I never planned before starting the coursework, I just started writing the code. This led to many bugs and having to rewrite a lot of simple things. This led to many 'weirdly' placed if and else statements. This function took me a little while to figure out, due to it either blocking the up, down and select button presses. To block even the protocols and sometimes even overload the display. However, my final Scroll function works perfectly under the desired requirements.

Another flaw I believe I had was my time management, I started this assignment a week after it was posted and due to my lack of knowledge of Arduino, I spent lots of time debugging and learning the syntax and logic in the language leading to it taking me almost a month to finish the coursework, not including the extensions. A way I would like to improve my work would be to use more functions and formulate a plan with timed incremental deadlines that I would impose on myself as this would improve my code and would increase my confidence in my work. Another improvement would be to finish the challenging HCI extension.

5 UDCHARS

```
//From the given custom character generator website on the lab week 5
byte upArrow[8] = { //defining 2 custom characters for the up and down
    0b00100,
    0b01110,
    0b10101,
    0b00100,
    0b00100,
    0b00100,
    0b00100,
    0b00100
};

byte downArrow[8] = {
    0b00100,
    0b00100,
    0b00100,
    0b00100,
    0b00100,
    0b00100,
    0b10101,
    0b01110,
    0b00100
};
```

Shown is the byte array for the up and down arrow

```
if (account > 0) {
    lcd.setCursor(0, 0);
    lcd.write((uint8_t)0);
}
if (account < agentCount - 1) {
    lcd.setCursor(0, 1);
    lcd.write((uint8_t)1);
}
```

Shown above is printing of the custom character to the lcd if there is an account above or below it in the array.

The code I used was from the website provided in the lab sessions, to form the up and down arrows. I then replaced the “V” and “^” with the 2 custom characters in my displayAgent() function. This made little to no difference to my FSM.

6 FREERAM

```
#include <MemoryFree.h>
```

Here I include the Memory free library

```
if (!selectButtonPressed && millis() - start > end)
  selectButtonPressed = true;
  lcd.clear();
  lcd.setBacklight(5);
  lcd.setCursor(0, 0);
  lcd.print("F421005");
  lcd.setCursor(0, 1);
  lcd.print(freeMemory());
```

Above is the code used to display the free memory to the lcd screen when the select button is held.

Using the Free RAM I was able to observe which functionality and processes in my system that would take up the most memory. Using this I was able to prioritise my memory allocation.

7 SCROLL

```
void scrollingTitle(int titleCount) { // attempt 7 at effectively scrolling the title without blocking thhe buttons being pressed

    String title = agents[titleCount].title + " ";

    if (title.length() > 7) {
        if (millis() - scrollTime[titleCount] >= 500) { // So scroll time is 2 characters per second
            lcd.setCursor(9, 1);
            for (int i = 0; i < 7; i++) {
                lcd.print(title[(scrollCount[titleCount] + i) % title.length()]);
            }
            scrollCount[titleCount] = (scrollCount[titleCount] + 1) % title.length();
            scrollTime[titleCount] = millis();
        }
    } else {
        lcd.setCursor(9, 1);
        lcd.print(title);
    }
}
```

above is the function I created to display the title as scrolling

```
if (livestate == main && agentCount > 0 && !selectButtonPressed || currentMenu == PENMenu || currentMenu == SalaryMenu || currentMenu == TaxMenu) {
```

```
    scrollingTitle(currentAccountNum);
```

I use this line of code to have the title to be scrolling without blocking proticals as I had many problems with this particular extension.

I first define a title variable with an extra space to make it look more presentable. Then in an if statement only titles with a length greater than 7 will scroll. For those that scroll set the speed to 1 character every half a second and scroll the title. 1 flaw in this is that I have the scrollingTitle() function constantly being run so it can sometimes overload the LCD and scroll a title even if no information is displayed.

```

if (title.length() < 7) {

    //Serial.print("DEBUG:  ");
    lcd.setCursor(9, 1);
    lcd.print(title);
}
else
for (int scroll = 0; scroll < title.length(); scroll++) {
    lcd.setCursor(9, 1);
    String text = title.substring(scroll, scroll + 7);
    lcd.print(text);
    delay(400);
    Serial.print("DEBUG:  ");
    if (text.length() < 8) {
        lcd.print(title.substring(0, 8 - text.length()));
        delay(400);
    }
}
}
}

```

This was an early attempt at my scroll function where I did not use the `millis()` function, this code led to blocking of the up and down buttons as well as the protocols. And would not scroll properly

8 HCI

```
enum Menu {
    Default,
    PENMenu,
    SalaryMenu,
    TaxMenu
};
Menu currentMenu = Default;

bool rightButtonPressed = false;
```

This is the enum I used to define the different Menus I would use.

```
double taxCalculation(double salary, int account) {

    double tax = 0;
    double PENContribution = salary;

    if (Pension(account)) {
        PENContribution = salary * 0.061;
    }
    double taxedSalary = salary - PENContribution;

    if (taxedSalary > 125140) {
        tax += (taxedSalary - 125140) * 0.45;
        taxedSalary = 125140;
    }
    if (taxedSalary > 50271) {
        tax += (taxedSalary - 50271) * 0.4;
        taxedSalary = 50271;
    }
    if (taxedSalary > 12570) {
        tax += (taxedSalary - 12570) * 0.2;
    }
    return tax / 12;
}
```

This is the list of calculations I made to work out the tax required

```

if (currentMenu == SalaryMenu) {
    lcd.setCursor(8, 0);
    lcd.print(agents[account].salary / 12);

} else if (currentMenu == TaxMenu) {
    lcd.setCursor(8, 0);
    lcd.print(taxCalculation(agents[account].salary, account), 2);

} else if (currentMenu == PENMenu) {
    if (!agents[account].PEN) {
        lcd.clear();
        return;
    } else {
        lcd.setCursor(8, 0);
        lcd.print(agents[account].salary);
    }
} else if (currentMenu == Default) {

    lcd.setCursor(8, 0);
    lcd.print(agents[account].salary);
}
}

```

This line of code is within my displayAgent() function to give the correct calculation based on the menu it is in.

```

if (buttons & BUTTON_RIGHT) {
    if (!rightButtonPressed) { // when pressing right system scrolls throught differnt Menus
        rightButtonPressed = true;

        if (currentMenu == Default) {
            lcd.clear();
            lcd.setBacklight(5);
            lcd.setCursor(0, 0);
            lcd.print("Pension Menu");
            delay(1000);
            lcd.clear();
            currentMenu = PENMenu;
        } else if (currentMenu == PENMenu) {
            lcd.clear();
            Serial.print("DEBUG: ENTER SAL MENU");
            lcd.clear();
            lcd.setBacklight(6);
            lcd.setCursor(0, 0);
            lcd.print("Salary Menu");
            delay(1000);
            lcd.clear();
            if (Pension(currentAccountNum)) {
                currentMenu = SalaryMenu;
            }
        }
    }
}

```

The button presses to go through menus, with a special screen to indicate which menu it is currently in.

```
bool Pension(int account) {  
    return agents[account].PEN;  
}
```

The Boolean function that checks accounts pension status

```
while (currentAccountNum < agentCount - 1 && currentMenu == PENMenu && !Pension(currentAccountNum))  
    //Serial.print("DEBUG:          ");  
    currentAccountNum++;
```

I use this code to skip accounts with NPEN status while in the pension menu

The first part of the code I added was the enum Menu with the different menus required such as salary etc. I then added to the displayAgents() function different lines of code concerning the salary. i.e. in the salary menu I divided the salary by 12 to get the monthly salary. In the tax menu, I added a taxCalculation() function to calculate tax. Also, I created the Pension() function to check if an account was on Pension or not via assigning a boolean value to it. As well as the displayMenu() function which contained code on what to do when the menu is entered. I implemented the use of the right button so the different menus could be entered.

This is the extent of what I have executed so far. Due to the large amount of debugging I have not yet finalized the HCI extension.